Data Structures and Algorithms I: Assignment 2 Solutions

Part A: Algorithmic Complexity Analysis

A1. Algorithm X Complexity

Algorithm X has a worst-case time complexity of ***O(N^2)***. This is because it uses nested loops to compare every pair of records in the array, driving the quadratic cost.

A2. Algorithm Y Complexity

Algorithm Y has a worst-case time complexity of ***O(N)***. It is faster than Algorithm X because it uses a hash table (the ***seen*** dictionary) for ***O(1)*** average-time lookups to find the complement of the target. The space trade-off is ***O(N)*** additional space for storing the hash table.

A3. Algorithm Z

This algorithm is Insertion Sort. Its best-case time complexity is ***O(N)*** (when the array is already sorted), and its worst-case time complexity is ***O(N^2)*** (when the array is in reverse order).

A4. Operations Table

Complexity Class	Approx. Operations at N=10^6	Rank (1=fastest)
O(1)	1	1
O(log N)	~20	2
O(N)	1,000,000	3
O(N log N)	~20,000,000	4
O(N^2)	1,000,000,000,000	5

A5. Recursive Function

The complexity is ***O(2^N)*** because the function branches into two recursive calls at each step, creating an exponential recursion tree. To reduce this to ***O(N)*** time and ***O(1)*** space, use an iterative approach with two variables to track the last two calculated values.

Part B: Arrays and Two-Pointer Technique

B1. Binary Search

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
```

```
        mid = (low + high) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: low = mid + 1
        else: high = mid - 1
    return -1
```

Best-case: ***O(1)*** (target at middle). Worst-case: ***O(log N)*** (repeatedly dividing the search space).

B2. Find Pair with Sum

```
def find_pair_with_sum(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        current_sum = arr[left] + arr[right]
        if current_sum == target: return (arr[left], arr[right])
        elif current_sum < target: left += 1
        else: right -= 1
    return None
```

For target 13 in the provided catalogue, a pair does not exist.

B3. Rotate Array

```
def rotate_array(arr, k):
    n = len(arr)
    k %= n
    def reverse(start, end):
        while start < end:
            arr[start], arr[end] = arr[end], arr[start]
            start, end = start + 1, end - 1
    reverse(0, n - 1)
    reverse(0, k - 1)
    reverse(k, n - 1)
```

B4. Amortised O(1)

Python lists are arrays that occasionally double in size when full, requiring an ***O(N)*** copy operation. However, since these resizes happen infrequently, the cost is spread out, resulting in an amortised constant time per append.